

Agenda

Fundamental Terms and Concepts

Installing and setting up python

Literals, Variables, and Numeral Systems

Input and Output in Python (Console I/O)

Operators and Expressions

timings

9.30AM to 11.00AM – P1

15 min Break

11.15 to 1.30PM – P2

1 Hr Break

2.30 to 3.45 – P3

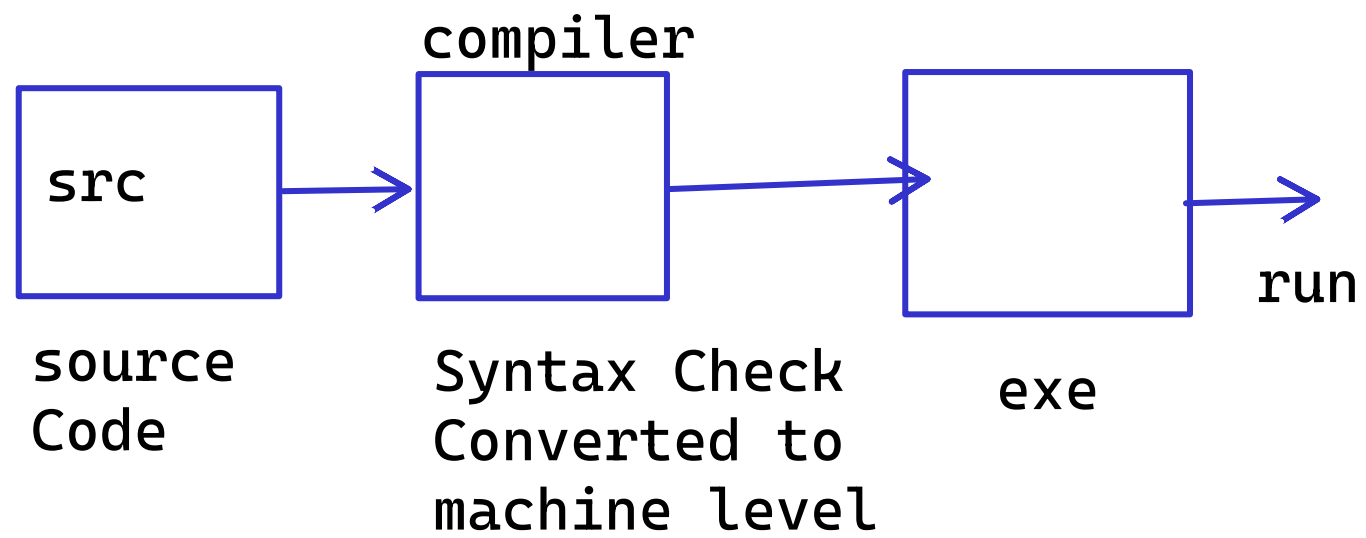
15 Break

4.00PM to 5.30PM – p4

Programming Languages

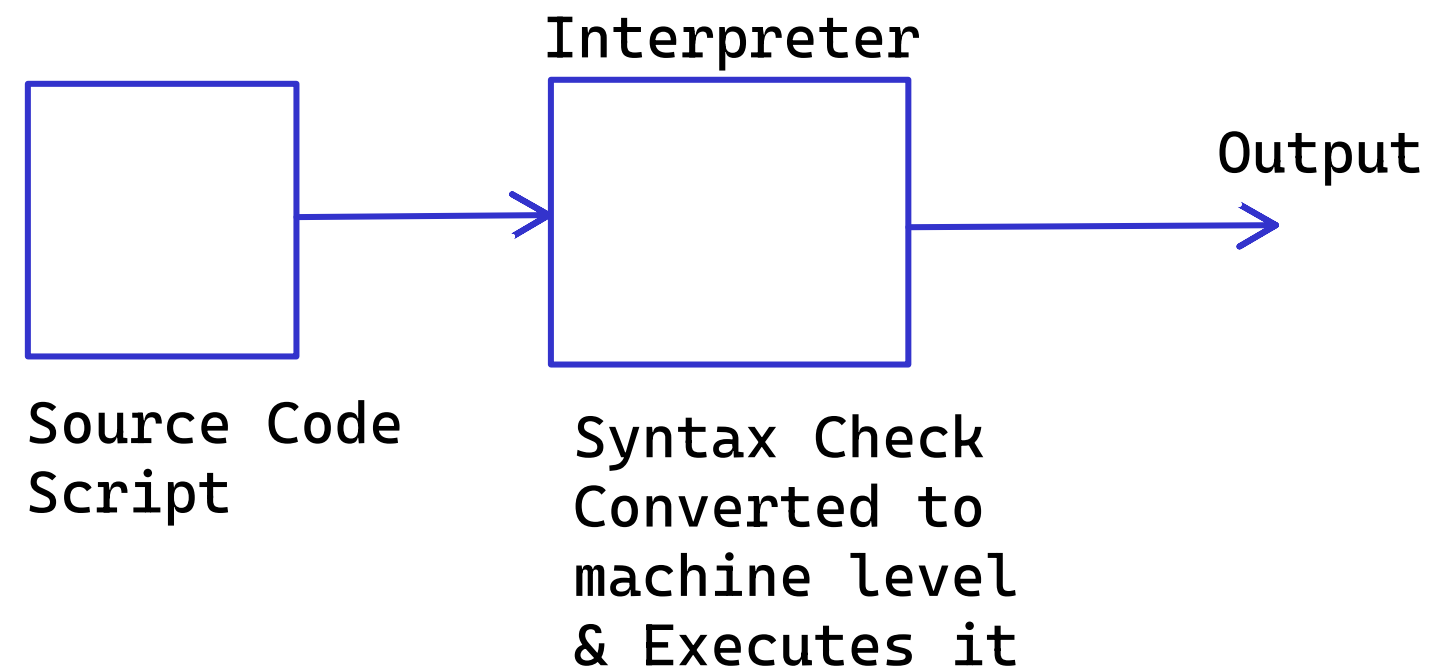
- Digital 1/0
- High Level instructions to Machine level Code (1/0)

Compiled Languages



Eg: C/C++/C#/Go/Rust

Interpreted Languages



Command by command

Eg: Python/JavaScript.....

Compiled Language

- More Faster
- Granular Control
- Takes hardware capability
- Secure compared to interpreted
- user does not need any additional software to use
- Takes time to compile
- Advanced/ Expertise needed to develop secure app

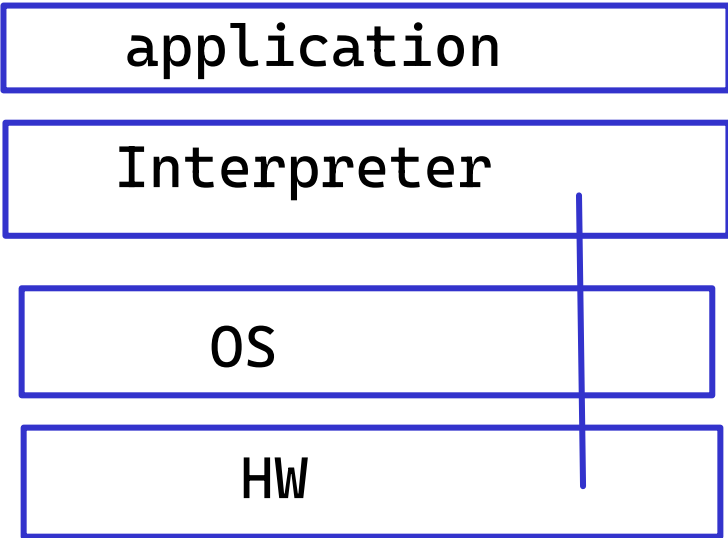
Interpreted Language

- More beginner Friendly
- less control compared to compiled
- slower than compiled
- Interpreter needed to run the applications
- Source code is available as a script

Python Programming Language

- Interpreted Language
- General Purpose Language
- Open Source Language
- Portable language / Platform Independent / WORA
- Dynamicaly typed language (later)

Guido Van Rossum
89-91
C/C++/Java



Application that you write using python can be run on any of these

Python Interpreter

Win/Lin/Mac	Win/Lin/Mac	Win/Lin/Mac
64bit	32bit	16bit

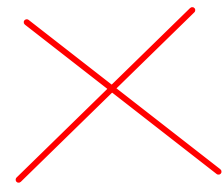
without modifications

Python Versions

2.x

3.x (latest)

2.x



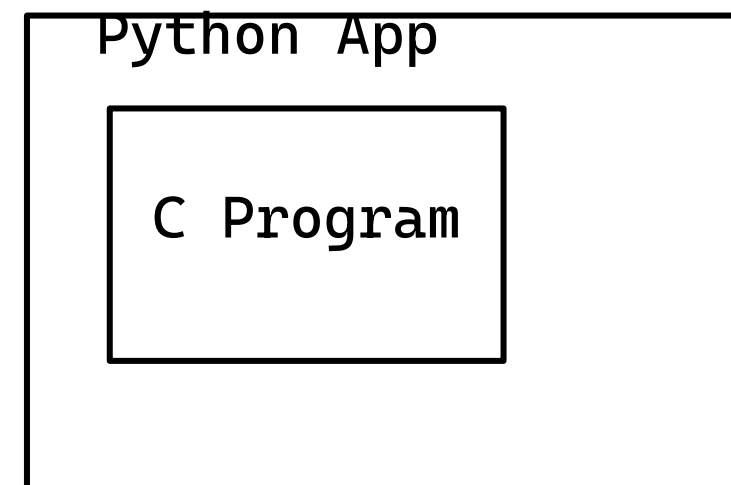
3.x

3.9.27 — 3.12.13 — 3.15.11

Python Interpreters

- Cython
- Jython
- RPython
- Pypi

Cython - C Python



Software needed to use / Develop python Programs

1. Python Interpreter
2. Text Editor – Notepad++, Atom, Sublime, VSCode, VIM, Geany, gedit, nano, emacs.....

Myth: Python needs big environments to develop & use

Interpreter+TextEditor+Misctools (IDE) – Integrated Development Environment

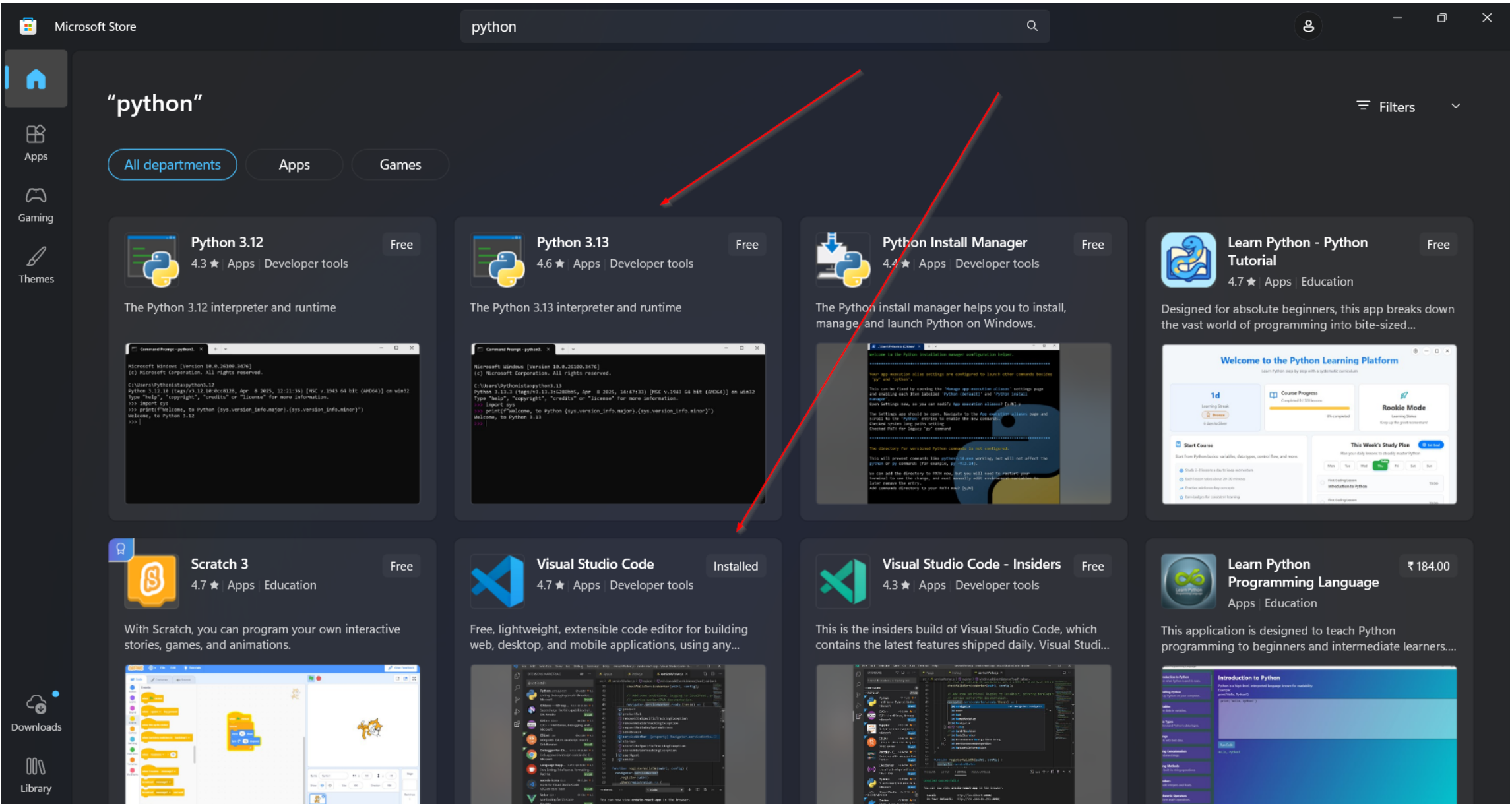
Pycharm, Spyder, jupyter Notebook, Conda env

recommendation:

If you are a beginner – Interpreter+VSCODE (withoutAI)

Installation of Softwares – Python, VSCODE

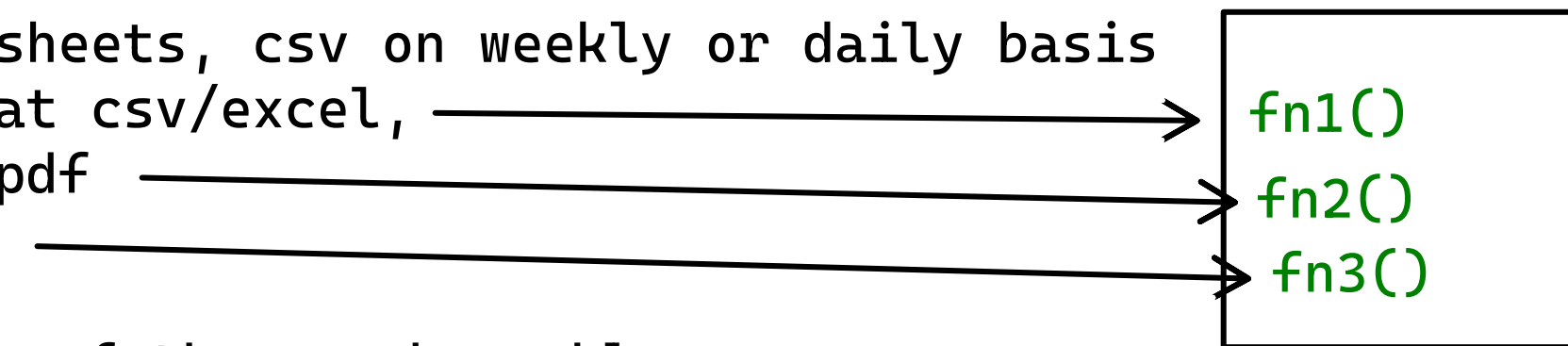
Lab: Installing Softwares



How to approach

You are a Consultant,
you get data in excel sheets, csv on weekly or daily basis
You have to process that csv/excel, _____
generate a reports in pdf _____
send it to management. _____

Single/Complete Solution



fn1()
fn2()
fn3()

Find Solutions for each of these sub problems

You have to process that csv/excel

Input: Excel/Csv

Output: Excel/Csv

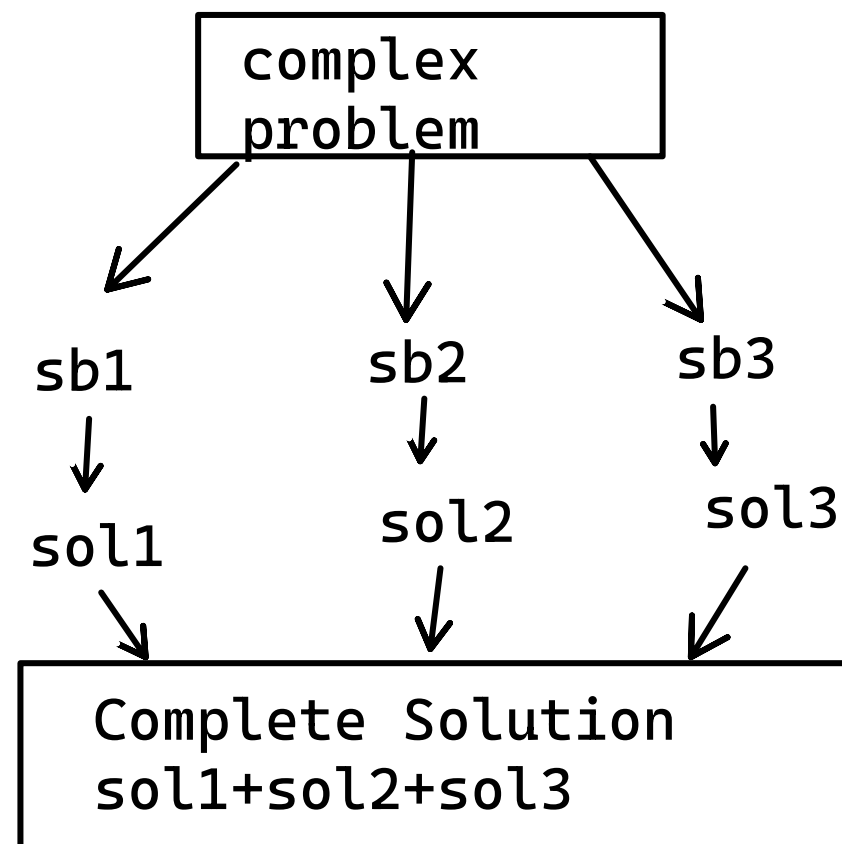
Logic: Cleaning of Data - Findout missing data & remove those
Data Format Correct
Date Format Correct

Programming in general

1. Why was that language created (Base paradigm) – Approach to problem
2. How to store Data Temporarily
3. How to Do Conditions
4. How to repeat (Loops / Functions)

Python Programming

1. Functional Approach – Solutions modeled in terms of Functions

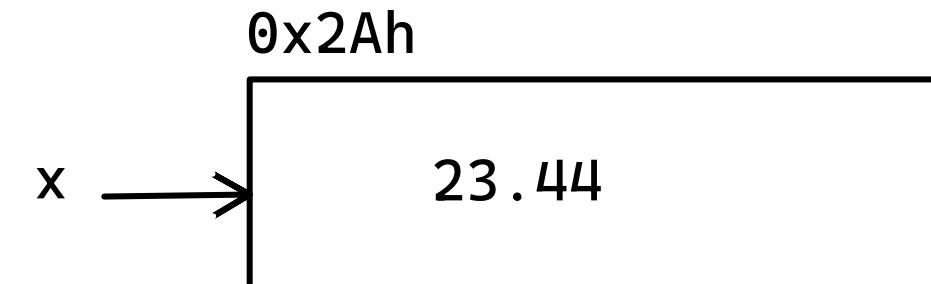


Python is a functional programming but also support Object oriented. Its an object based programming language(internal functionality is classes & objects)

2. How to store data temporarily(Variables)

Vishwas
23
23.45

x	=	23.44
label	Assignment	data
variable	operator	
identifier		



Dynamically Typed Language – You do not specify the data type before using a variable
Interpreter automatically detects it

Rules:

- alpha numeric & _ only
- Should not start with a number
- should not be a keyword

Conventions:

- Use meaningful names
- use snake_case

number_of_words

```

>>> print(keyword.kwlist)
['False', 'None', 'True', 'and', 'as', 'assert', 'async', 'await', 'break', 'class', 'continue', 'def', 'del', 'elif', 'else', 'except', 'finally', 'for', 'from', 'global', 'if', 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']

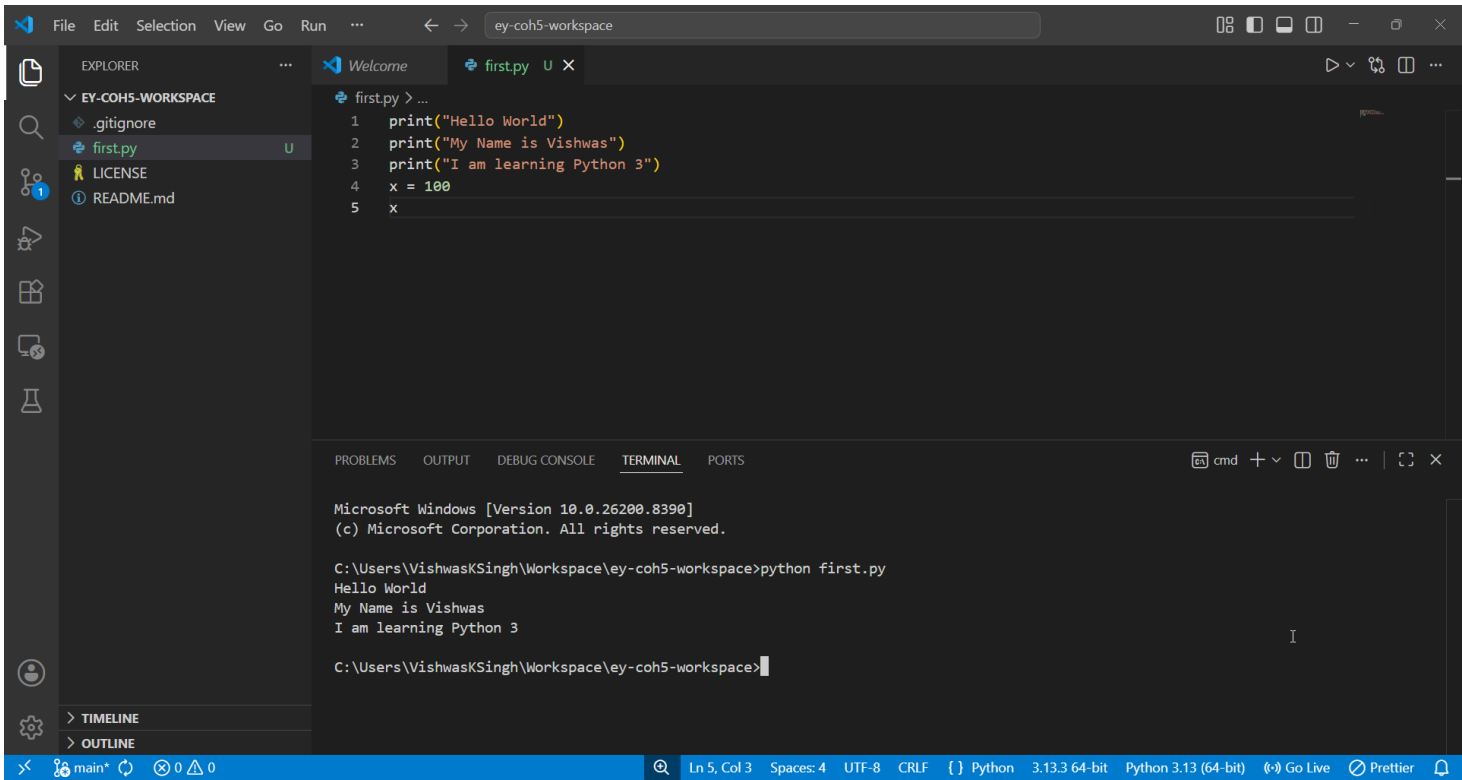
```

How to interact with Python

- Interactive mode

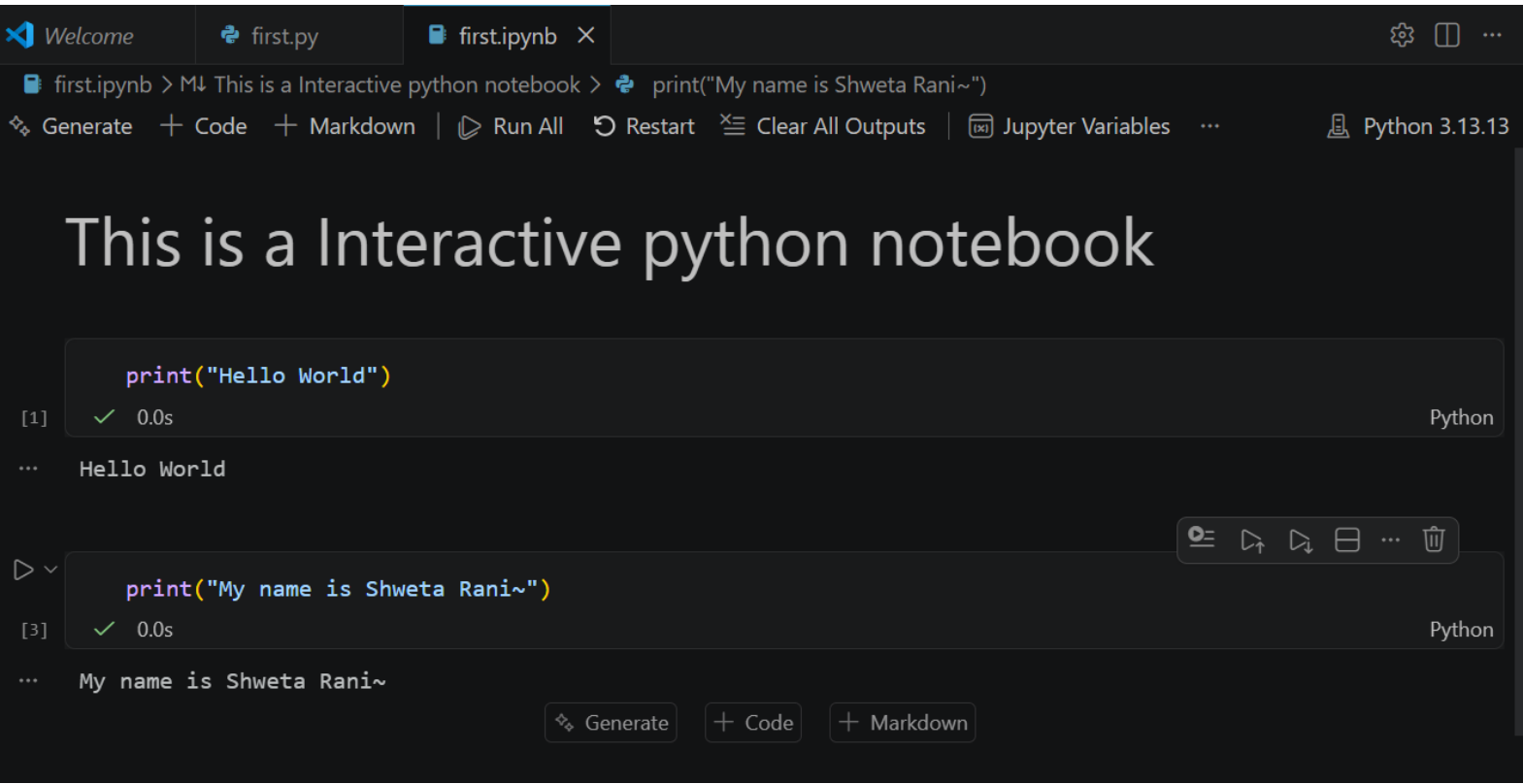
```
C:\Users\VishwasKSingh>python
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr  8 2025, 14:47:33) [MSC v.1943 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> print("Hello World")
Hello World
>>> 2+3
5
>>> 4+5
9
>>> x = 10
>>> x
10
>>> quit()
```

- Script Mode



- working of snippets of code

- Notebook Mode



Data Types in Python – Numbers

```
>>> x = 100
>>> x1 = "100"
>>> type(x)
<class 'int'>
>>> type(x1)
<class 'str'>
>>> x3 = 23.45
>>> type(x3)
<class 'float'>
>>> x4 = 7812731278378173917927419273912387391283919712491287519274192485712948120381724912812032184102384124182491271249237424913712
>>> x4
7812731278378173917927419273912387391283919712491287519274192485712948120381724912812032184102384124182491271249237424913712
>>> x5 = 2123214234123.12312423141902749837498237498237498237498
>>> x5
2123214234123.123
>>> x6 = 12.12323345465765678798980978078567457564564
>>> x6
12.123233454657656
```

```
>>> x8 = 10_00_000
>>> x8
1000000
>>> x9 = 0b10101111
>>> x9
175
>>> type(x9)
<class 'int'>
>>> x10 = 0x1AB
>>> x10
427
>>> x11 = 0o123
>>> x11
83
>>> |
```

```
>>> l1 = 123.45e10
>>> l1
1234500000000.0
>>> 0.12345e4
1234.5
>>> l2 = complex(2,1)
>>> l2
(2+1j)
>>>
```

Strings

```
>>> name = "Vishwas K Singh"
>>> l6 = f"Hi my name is {name}"
>>> l6
'Hi my name is Vishwas K Singh'
>>> salary = 20000.456789
>>> l6 = f"Hi my name is {name}, My salary is {salary:.2f}"
>>> l6
'Hi my name is Vishwas K Singh, My salary is 20000.46'
>>> |
```


Taking Input from the user

- Whatever input you take from the user is always a string
- its the job of the developer to convert to right data type

```
>>> name = input("Enter your name: ")
Enter your name: vishwas k singh
>>> name
'vishwas k singh'
>>> num = input("Enter a number: ")
Enter a number: 34
>>> num
'34'
>>> type(num)
<class 'str'>
>>> num = int(num)
>>> num
34
>>> type(num)
<class 'int'>
>>> num = int(input("Enter a number: "))
Enter a number: 67
>>> type(num)
<class 'int'>
>>> num
67
>>>
```

```
2
3 name = input("Enter your name: ")
4 num1 = int(input("Enter first Number: "))
5 num2 = int(input("Enter second Number: "))
6 print(f"Welcome {name}, the sum of numbers entered is: {num1+num2}")
```

PROBLEMS

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

JUPYTER

```
C:\Users\VishwasKSingh\Workspace\ey-coh5-workspace>python 01-basics.py
Enter your name: vishwas
Enter first Number: 23
Enter second Number: 34
Welcome vishwas, the sum of numbers entered is: 57
```

1. 0 is always False. Non-Zero is always True

Short Circuit Evaluation:

- And Expression
out of 2 expressions (expr1 and expr2)
expr 2 is evaluated only when expr1 is True
- Or Expression
expr 2 is not evaluated at all when expr1 is already True

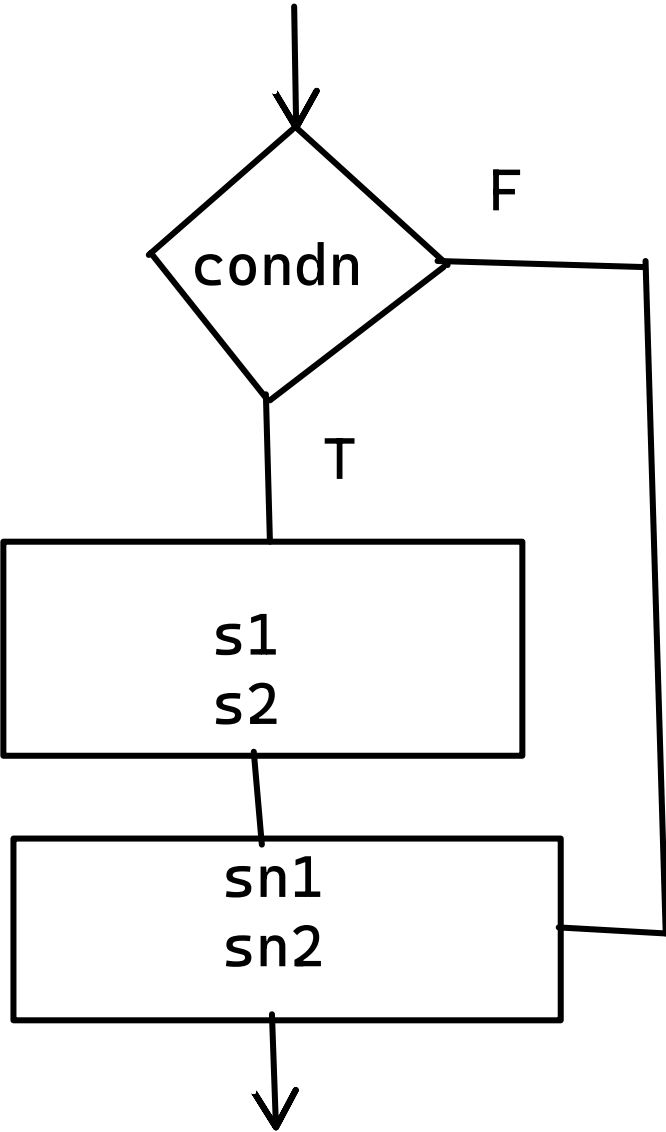
```
>>> 3 and 2
2
>>> 5 and 7
7
>>> 6 and 3%2
1
>>> 6 and 3/2
1.5
>>> 2 or 3
2
>>> 3/2 or 3
1.5
>>>
```

Guard Pattern

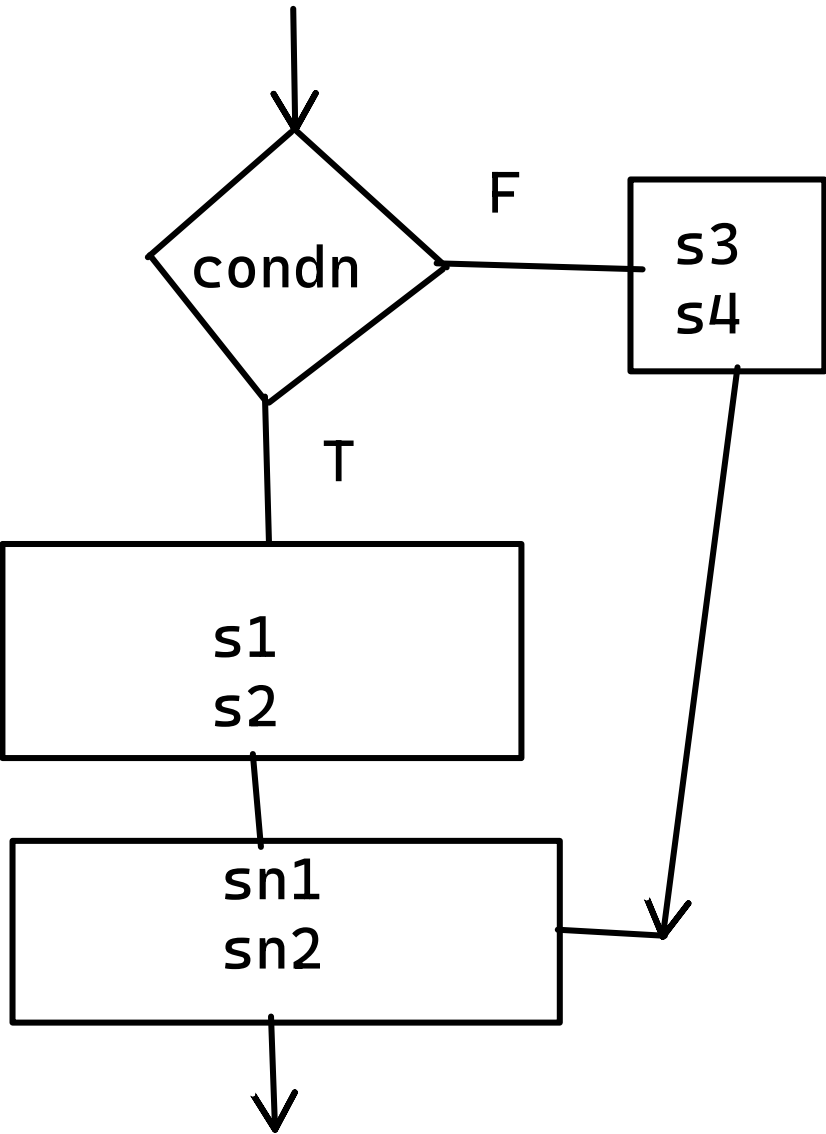
```
>>> x = 10
>>> y = 2
>>> x / y
5.0
>>> y = 0
>>> x / y
Traceback (most recent call last):
  File "<python-input-113>", line 1, in <module>
    x / y
    ~^~
ZeroDivisionError: division by zero
>>> y != 0 and x / y
False
>>> y = 2
>>> y != 0 and x / y
5.0
>>> |
```

Conditional

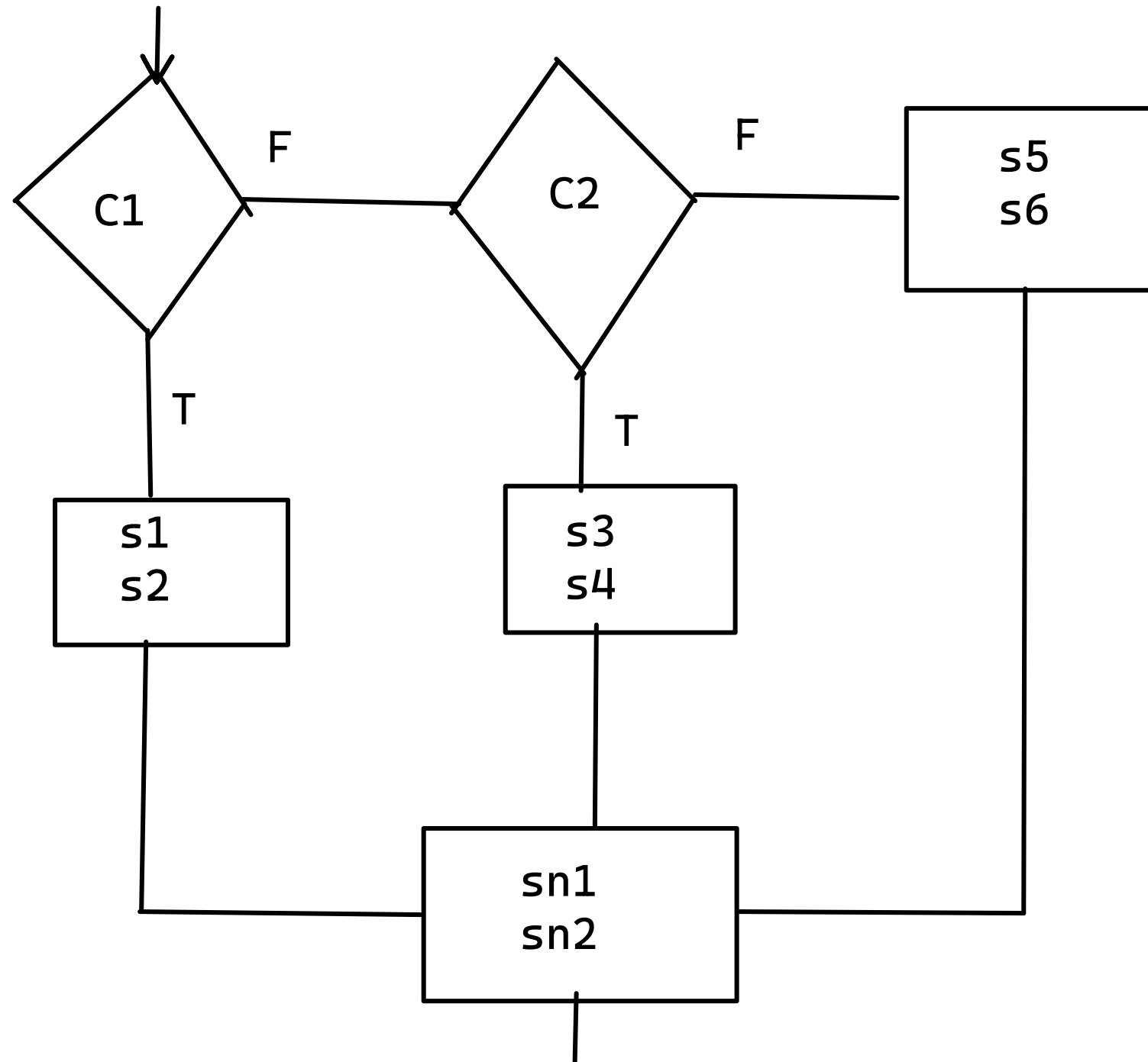
```
if condition:  
    s1  
    s2  
sn1  
sn2
```



```
if condn:  
    s1  
    s2  
else:  
    s3  
    s4  
sn1  
sn2
```




```
if condn1:  
    s1  
    s2  
elif condn2:  
    s3  
    s4  
else:  
    s5  
    s6  
sn1  
sn2
```



nested if

```
if condn:  
    if condn:  
        s1  
        s2  
    else:  
        s3  
        s4  
else:  
    s5  
    s6
```

Always note the order of condition check

Grade System

90 >	A
80 > < 90	B
70 > < 80	C
< 70	D

```
x = 75
if x > 50:
    print("x>50")
elif x > 30:
    print("x>30")
elif x > 70:
    print("x>70")
else:
    print("x<30")
```

```
marks = 80
if marks > 90:
    print("GA")
elif marks > 80 and marks < 90:
    print("GB")
elif marks > 70 and marks < 80:
    print("GC")
else:
    print("GD")
```

What all statements will be printed

`x>50`

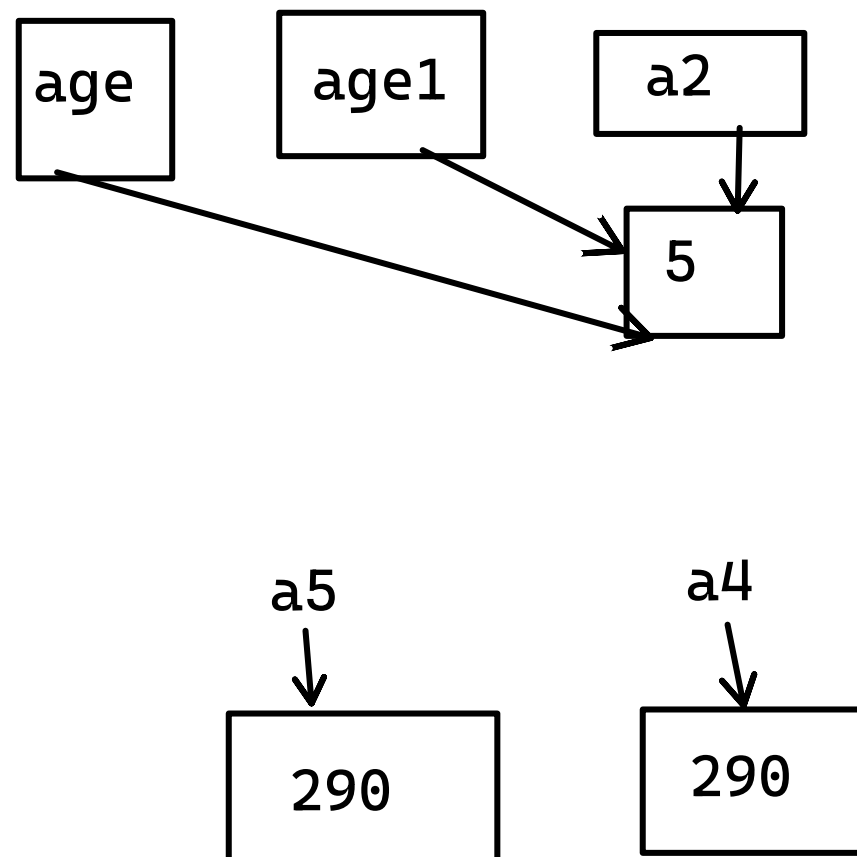
Print Statements

```
name = input("Enter second number: ")  
print(f"Welcome {name}, the sum of numbers entered is: {num1+num2}")  
print("Welcome", name, " the sum of numbers entered is: ", num1+num2)  
print("Welcome "+ name + " the sum of numbers entered is: " + str(num1+num2))
```

```
print("Hello", end="**")  
print("Vishwas", end="*")  
print("Singh")
```

```
>>> print("Hello"*3)  
HelloHelloHello
```

When you are storing integers
if the value is < 255 and multiple variables are storing the same value
then they will be pointed by same object



```
>>> age = 5
>>> id(age)
140722280866856
>>> age1 = 5
>>> id(age1)
140722280866856
>>> age2=250
>>> a3=250
>>> id(age2)
140722280874696
>>> id(a3)
140722280874696
>>> a4=290
>>> a5=290
>>> id(a4)
2709958056688
>>> id(a5)
2709958056016
>>>
```

```
>>> bin(23)
'0b10111'
>>> hex(23)
'0x17'
>>> oct(23)
'0o27'
>>> ord('a')
97
>>> chr(98)
'b'
```

```
>>> v1 = 0b1011
>>> v1
11
>>> type(v1)
<class 'int'>
>>> bin(v1)
'0b1011'
>>> v1
11
>>> print(v1)
11
>>> |
```

Python Programming Day 1

- What is Python
- Variable, Data Types, operators, Conditionals, Expressions

What to expect for tomorrow

- Loops, Data Structures - Lists, Strings, Dict, Tuples, Sets
- functions

Practice Problems:

1. Build a Temperature Converter
2. Build a Simple Interest Calculator
3. Brokerage Calculator
4. SIP vs Lumpsum Calculator (*)